COMP5047 Task 2: Analysis and Specify Software Quality Requirements

---

Subsystem: USU Operation System
Functional Requirement: FR-USU-1: Management of Unions

A quality requirement is a non-functional requirement that constrains how software should meet its functional requirements or how it should be developed (Pressman & Maxim).

This document defines the software quality requirements for FR-USU-1: Management of Unions in the University Student Unions system. FR-USU-1 includes key functions such as approving union membership, updating union registration data, and terminating memberships. These actions involve sensitive organisational and personal data, decision-making authority, and nationwide usage, making clear quality requirements essential to ensure the system is secure, performant, reliable, and scalable.

Each quality requirement specifies measurable and testable criteria, including a metric, unit of measurement, target value, and precision. The target value defines the desired level of performance, making the requirement falsifiable and objectively verifiable. Precision specifies the granularity of measurement, ensuring consistent and repeatable testing. This approach guarantees that all requirements are clear, unambiguous, and verifiable, enabling systematic evaluation of the system's performance, reliability, security, and scalability (Kazman; ISO/IEC 25010).

Quality Requirements

Scenario 1: Login and Authentication/ Privilege Access

Quality Attribute	Requirement statement	Metric	Unit	Precision	Target Value	How to Measure/ Test
Security	All USU officers must login with unique username/password (min 12 chars) + MFA, invalid attempts denied, account locks after 3 failed attempts for 20 mins.	Unauthorised access attempts.	Attempts	1	0 successful	Attempt login with invalid credentials, verify lockout occurs, check audit log.
Performance	Login response time must be fast to avoid delay.	Response Time	Seconds	0.1	≤ 2	Automated timing of login requests under normal and peak times.
Reliability	System must correctly authenticate users on every attempt.	Authentication Success Rate	%	0.1	100	Test valid credentials multiple times, confirm system always authenticate correctly.
Scalability	System must support concurrent logins from all USU officers.	Max concurrent logins	Users	1	≥ 500	Load testing with 500+ concurrent login attempts.

Security in this context means preventing unauthorised users from accessing sensitive union data and system functions. Performance refers to how quickly the system responds to login requests. Reliability means the authentication process must work correctly every time without failure, and scalability means the system must support many officers logging in concurrently without degradation.

This scenario ensures that only authorised USU officers access FR-USU-1 functions. Security measures such as multifactor authentication (MFA), strong password policies, session management, and audit logging create a layered defence against unauthorised access, brute-force attacks, and insider threats, ensuring compliance with GDPR. Reliability ensures the system always remains available and functional, while performance and scalability requirements guarantee smooth operation even under peak load. All aspects are measurable and testable, e.g. login response time can be timed, authentication success can be tracked, and concurrent logins can be stress-tested.

Scenario 2: Approval of Membership (FR-USU-1a)

Quality Attribute	Requirement statement	Metric	Unit	Precision	Target Value	How to Measure/ Test
Security	Only authenticated USU officers can approve membership; maintained.	Unauthorised attempts logs	Attempts	1	0	Attempt approval with non-officer account, check logs for completeness.
Performance	Approval processing must be fast.	Approval time processing time.	Seconds	0.1	≤ 5	Measure time from submission to system confirmation.
Reliability	Approved memberships must be stored and accessible immediately.	Correct approvals	Count	1	100%	Approve sample memberships, verify correct data storage and retrieval.
Scalability	System must handle approval of up to 1000 memberships/hour. our	Memberships processed/h	Count	1	≥ 1000	Simulate bulk approvals under load, verify system handles expected volume.

Security in this context means only authorised USU officers can approve new unions, and all actions are fully traceable through audit logs. Performance refers to the speed at which membership approvals are processed, ensuring timely response even under heavy load. Reliability means that approved membership data is stored correctly and consistently without errors, guaranteeing correct account activation and role assignments. Scalability ensures the system can handle a high volume of membership approvals simultaneously, particularly during busy periods such as the start of the semester.

The approval process ensures that only verified unions participate in USU activities. Measures such as encryption and audit logging maintain confidentiality, integrity, and auditability, supporting accountability and compliance with data protection regulations (GDPR). All quality attributes are measurable and testable: approval processing time can be timed, correct storage of membership data can be verified, and system performance under peak load can be stress-tested to confirm scalability.

Scenario 3: Approval of Updates to USU Registration Data (FR-USU-1b)

Quality Attribute	Requirement statement	Metric	Unit	Precision	Target Value	How to Measure/ Test
Security	Only USU officers can approve updates, all updates logged.	Unauthorised updates	Count	1	0	Attempt update with non-officer account, verify logs capture all actions
Performance	Update approval must propagate changes quickly.	Update propagation time	Seconds	0.1	≤ 5	Measure time from approval to data visible in system
Reliability	Approved updates must reflect correctly without errors.	Accuracy of updates	%	0.1	100	Verify updated data against intended changes in test database
Scalability	System must support 500 unions updating concurrently.	Concurrent updates handled	Count	1	≥ 500	Simulate 500 concurrent updates, confirm system stability

Security in this context means only authorised USU officers can approve updates, and all changes are fully auditable through detailed logs. Performance refers to the speed at which approved updates propagate and become visible in the system, ensuring timely access to accurate information. Reliability means that updated data must be stored correctly and consistently without errors, preserving the integrity of union registration records. Scalability ensures the system can handle multiple unions submitting updates concurrently without degradation in performance.

Scenario 3 ensures that union registration data remains accurate, secure, and up to date. Encryption protects confidentiality, and logging enables traceability and accountability, supporting compliance with data protection regulations (GDPR). All quality attributes are measurable and testable: update propagation time can be timed, data accuracy verified against intended changes, and system performance under concurrent updates can be stress-tested to confirm scalability.

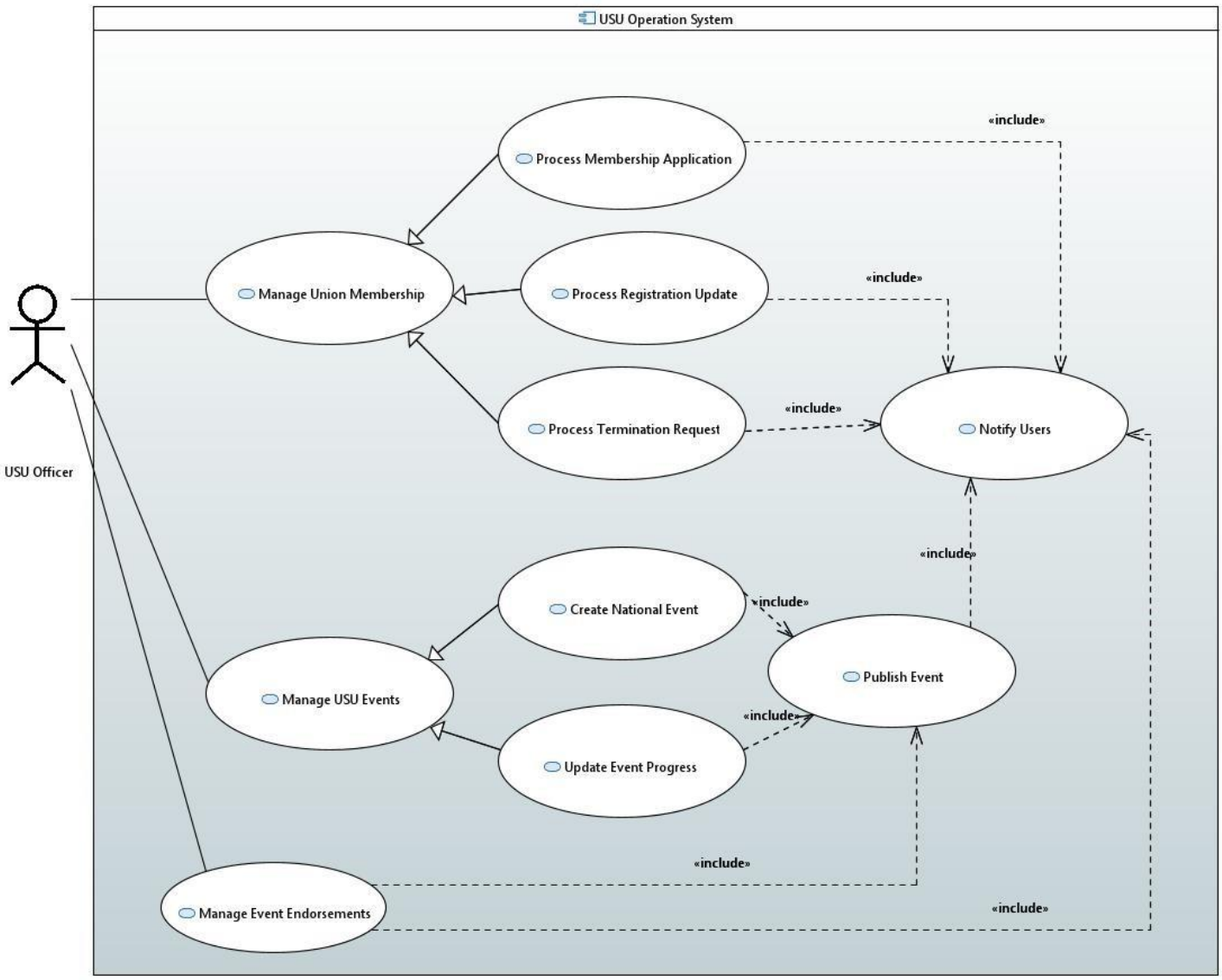
Scenario 4: Approval of Termination of Membership (FR-USU-1c)

Quality Attribute	Requirement statement	Metric	Unit	Precisi on	Target Value	How to Measure/ Test
Security	Only USU officers can approve terminations, accounts locked, logs maintained.	Unauthorised termination s	Count	1	0	Attempt termination with non-officer account, check audit logs
Performance	Termination requests processed promptly.	Termination processing time	Secon ds	0.1	≤ 10	Measure time from request approval to account deactivation
Reliability	Terminated accounts deactivated, data retained 1 month then deleted.	Deactivation & deletion success	%	0.1	100	Verify deactivated accounts cannot log in, check data deletion after 1 month
Scalability	System must handle 1000 concurrent terminations.	Concurrent termination s handled	Count	1	≥ 1000	Simulate 1000 termination requests concurrently, confirm system stability

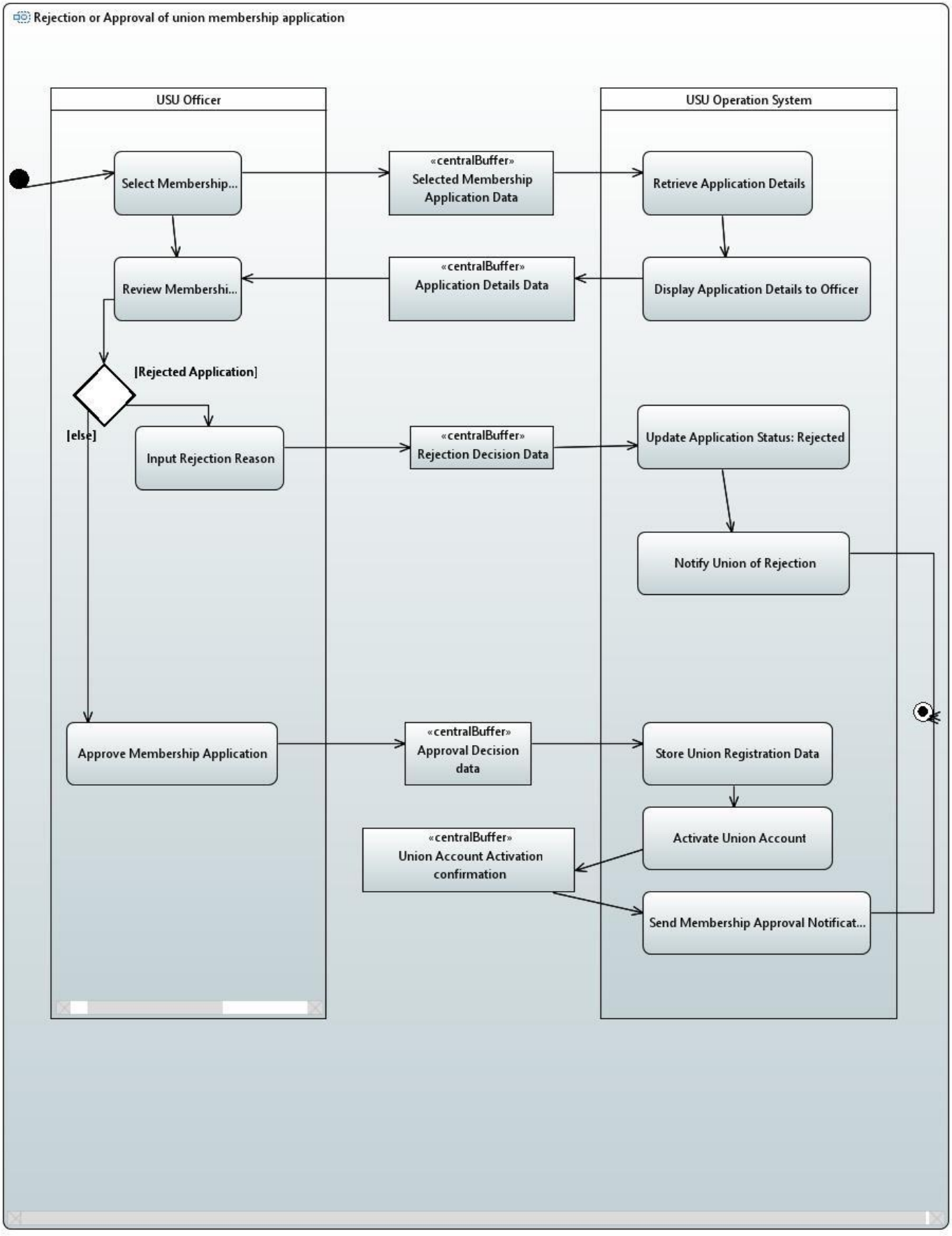
Security in this context means only authorised USU officers can approve terminations, and all termination actions are logged for traceability and accountability. Performance refers to the speed at which termination requests are processed and accounts deactivated, ensuring timely removal of inactive unions. Reliability means that terminated accounts are consistently disabled, retained securely for one month, and permanently deleted thereafter, preserving data integrity and compliance with GDPR. Scalability ensures the system can handle large volumes of terminations simultaneously without failure or performance degradation.

Scenario 4 guarantees that inactive unions are removed safely and securely. Temporary retention allows potential reactivation, while permanent deletion protects privacy and sensitive data. All quality attributes are measurable and testable: termination processing time can be timed, deactivation and deletion verified, and system behaviour under concurrent terminations stress-tested to confirm scalability. Logging provides a verifiable audit trail to support security and accountability.

Task 3a: Use Case Diagram USU-OS Subsystem

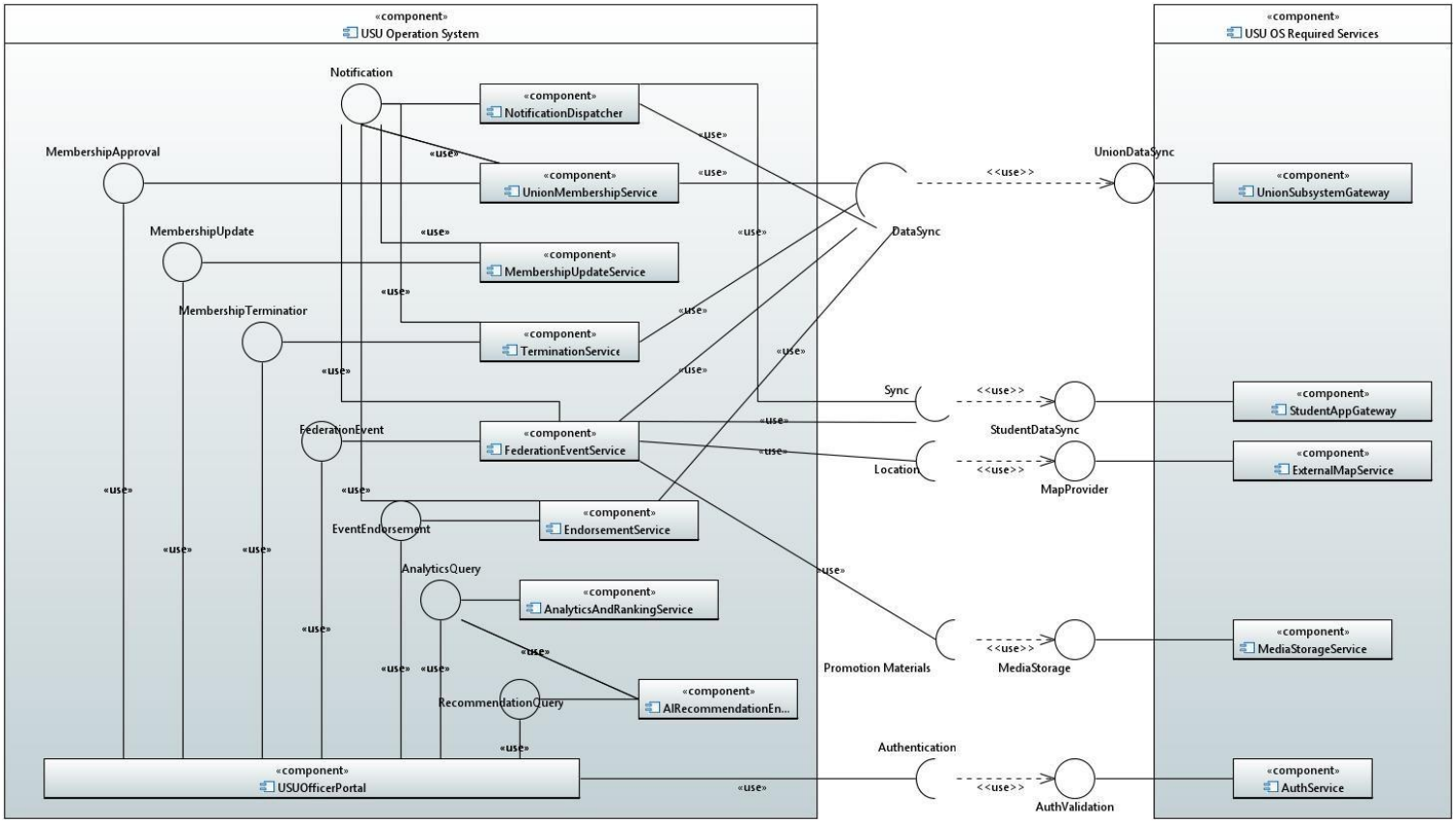


Task 3b: Activity Model: Processing Membership Application



The activity diagram models the functional requirement of processing membership applications. It covers how union officers are notified of their application outcome and the activation of union accounts if approved.

Task 4a: Architecture of the USU-OS Subsystem



Microservice and interface descriptions

UnionMembershipService

Component	*UnionMembershipService*
Description	Handles all membership related operations: retrieving applications from unionsubsystemgateway processing approvals or rejection decisions. Communicates decisions to external systems
Stereotype	<<microservice>>
Required Interfaces	UnionDataSync (get the applications to process), Notification
Provided Interfaces	MembershipApproval

Name	*MembershipApproval*	
Provider	UnionMembershipService	
Operation	Signature	getApplicationDetails(in unionId: String): string
	Function	Retrieves membership application data for fived union ID so officer cab review before making a decision.

Operation	Signature	approveMembership(in unionid: String, in officerId: String, in decisionNote: String) : String

	Function	Records an approval decision made by officer, validates the request and triggers downstream updates like account activation and notifications.
Operation	Signature	rejectMembership(in unionid, String, in officerId: String, in reason: String): String
	Function	Records and processes rejection decision, saves rejection record and triggers follow-up actions like notifications and audit loggin.

MembershipUpdate

Component	*MembershipUpdateService*
Description	Handles updates to existing union membership records and ensures relevant parties are notified.
Stereotype	<<microservice>>
Required Interfaces	Notification: used to send update alerts to relevant people.
Provided Interfaces	MembershipUpdate

Name	*MembershipUpdate*	
Provider	MembershipUpdateService	
Operation	Signature	getUpdateRequest(unionId: String): String
	Function	Retrieves pending update request for the specified union so the USU officer can review before making decision.
Operation	Signature	approveUpdate(in updateData: String, in officerId: String): String
	Function	Records approval of requested membership update, applies the updates details to union recird and triggers synchronisation and notification processes.
Operation	Signature	rejectUpdate(in updateData: String, in reason:String) :String
Funtion		Records a rejection decision, stores officers reason and informs relevant people that the update has not been accepted.

TerminationService

Component	*TerminationService*
Description	handles process of membership termination requests submitted by unions. Retrieves termination details, applies approval decisions, deactivates union account and coordinates all required system updates.
Stereotype	<<microservice>>
Required Interfaces	UnionDataSync, Notification
Provided Interfaces	MembershipTermination

Name	*MembershipTermination*	
Provider	TerminationService	
Operation	Signature	reviewTerminationRequest(in unionId: String): String
	Function	Retrieves the termination request submitted by union, including justification and requested termination date so UO can review details.

Operation	Signature	approveTermination(in unionid: String, in officerId: String, in decisionNote: String): String
	Function	Records the officer's approval decision, deactivates union account, intiates the one month retention period and triggers downstream synchronisation and notification processes.

FederationEventService

Component	*FederationEventService*
Description	Manages creation, publishing and progress updates of national USU federation events. Supports fr-usu-2 and fr-uo
Stereotype	<<microservice>>
Required Interfaces	UnionDataSync, Notification, StudentDataSync, MapProvider, PromotionMaterials
Provided Interfaces	FederationEvent

Name	*FederationEvent*	
Provider	FederationEventService	
Operation	Signature	createEvent(in eventData: String, in officerId: String): String
	Function	Creates new USU federation event using the provided event details. Returns unique event identifier once the event is successfully stored.
Operation	Signature	publishEvent(in eventId: String): String
	Function	Publishes event across all connected subsystems distributing details to unions and students and initiating notifications.
Operation	Signature	updateEventProgress(in eventide: String, in progress: String): String
	Function	Updates status or progress information for an existing federation event and synchronises the update with external systems.

EventEndorsement

Component	*EndorsementService*
Description	Handles workflow for unions requesting and submitting endorsements for federations events. Supports fr-uo-3 and fr-usu-2 collaboration workflow.
Stereotype	<<microservice>>
Required Interfaces	UnionDataSync, Notification
Provided Interfaces	EventEndorsement

Name	*EventEndorsement*	
Provider	EndorsementService	
Operation	Signature	requestEndorsement(eventide: String, unionid: String): String
	Function	Creates a new endorsement request from a union for a federation event, stores the request and synchronises it through UnionDataSync. Returns a requestId string so system can track the endorsement.

Operation	Signature	recordEndorsement(unionid: String, eventId: String, endorsementDecision: String): String
	Function	Records the union's endorsement outcome, saves the decision details and synchronises the update via UniondDataSync. Returns confirmationId string.

NotificationDispatcher

Component	*NotificationDispatcher*
Description	Handles delivery of all outgoing notifications from USU-OS subsystem to unions and sydents. Supports FR notification triggers and KF-1 real time updates.
Stereotype	Service
Required Interfaces	UnionDataSync, StudentDataSync,
Provided Interfaces	Notification

Name	*Notification*	
Provider	NotificationDispatcher	
Operation	Signature	notifyStudents(notification: String): void
	Function	Sends event updates and relevant information to students via studentdatasync like ticket updates, event updates etc.
Operation	Signature	notifyUnionOfficers(notification: String): void
	Function	Sends targeted messages to officers (e.g. endorsement requests, membership decisions) via uniondatasync
Operation	Signature	notifiyMembershipDecision(unionid: String, decisionId: String):void
	Function	Sends membership approval, rejection or termination decisions to the appropriate union via uniondatasync.

AnalyticsAndRankingService

Component	*AnalyticsAndRankingService*
Description	Provides system wide and union specific analytics to support USU operational decision making. Implements key performances measurement functions (KF-analytics). Retrieves performance, engagement and global statistical data.
Stereotype	Service
Required Interfaces	UnionDataSync, StudentDataSync
Provided Interfaces	AnalyticsQuery

Name	*AnalyticsQuery*	
Provider	AnalyticsAndRankingService	
Operation	Signature	getUnionPerformance(unionid: String): String

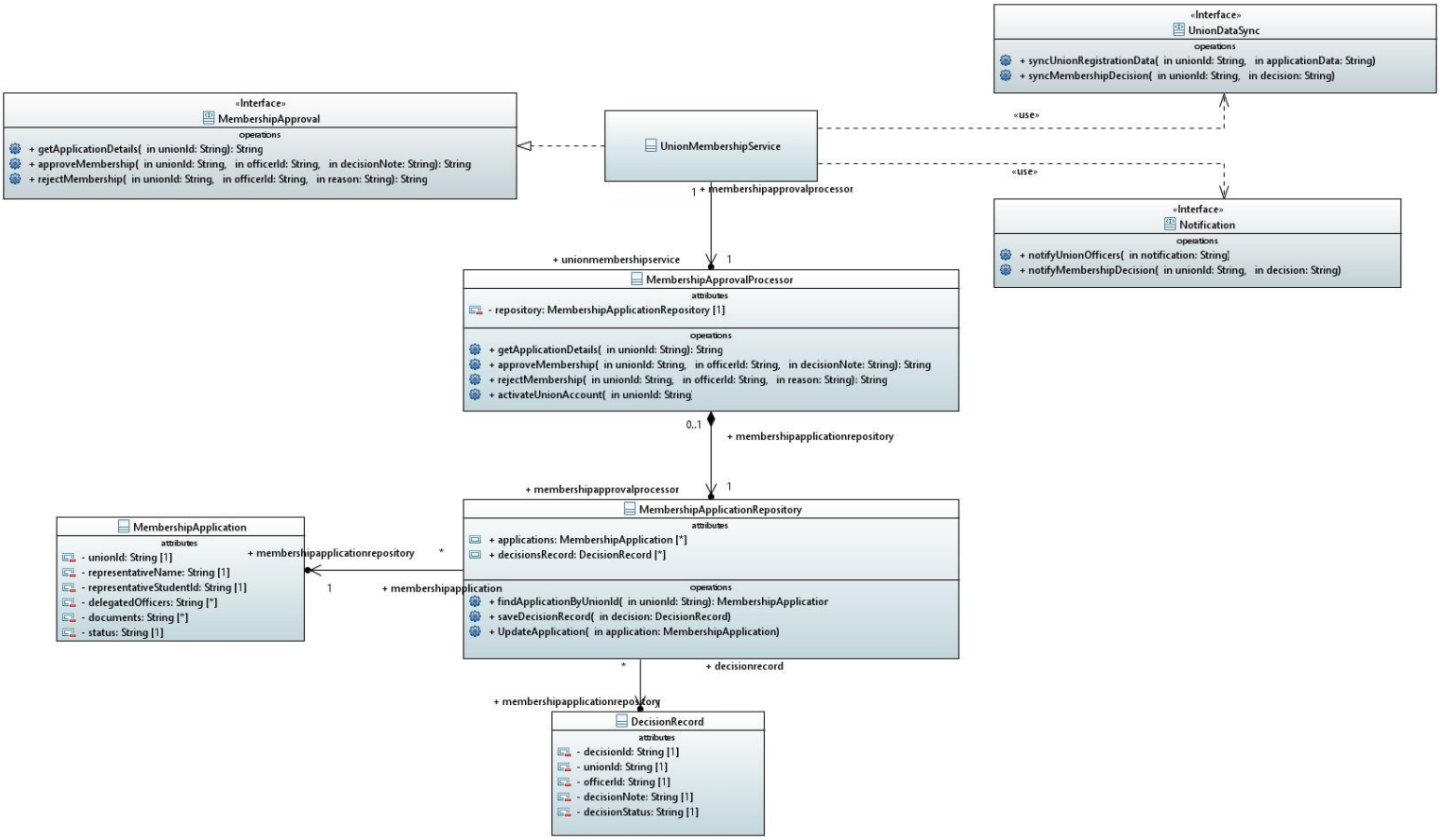
	Function	Retrieves analytics for specific union (member activity trends, event participation, contribution metrics). Used by USU officers to assess union engagement and compliance.
Operation	Signature	getEventEngagement(eventId: String):String
	Function	Returns event specific engagement analytics (attendanace, interest rates). Supports performance evaluation of USU federation events.
Operation	Signature	getSystemStatistics():String
	Function	Provides aggregated analytics across all unions and events, supporting high-level insights required for management reporting and long-term planning.

AIRecommendationEngine

Component	*AIRecommendationEngine*
Description	Provides information to improve decision making for UO and USU's. Uses analytics data to suggest promotion strategies, target unions and resource allocation. Support KF-AI optimisation.
Stereotype	Service
Required Interfaces	Analytics query
Provided Interfaces	recommendationquery

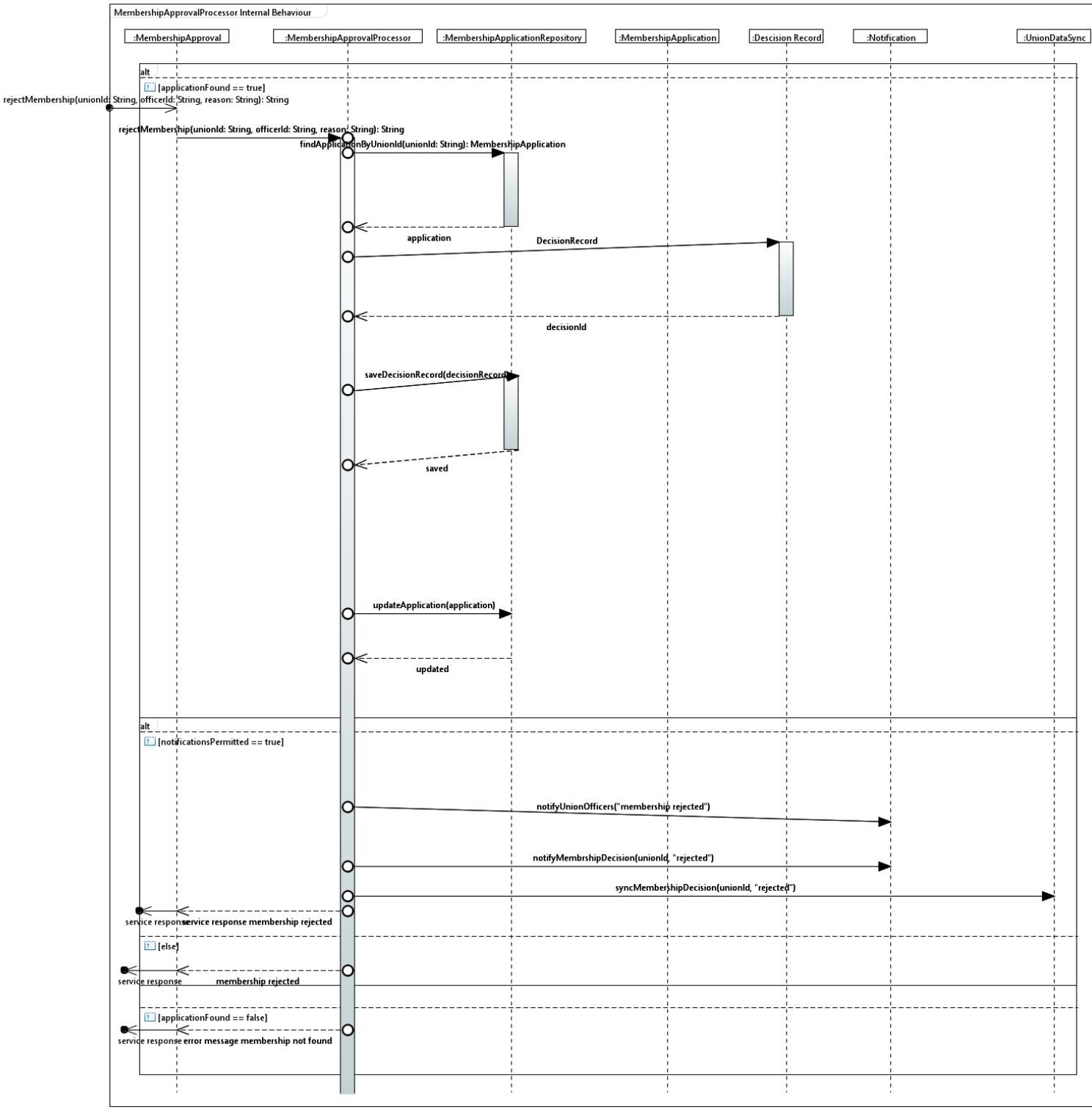
Name	*RecommendationQuery*	
Provider	AIRecommendationEngine	
Operation	Signature	recommendedPromotionsStrategies(unionid: String):String
	Function	Generates personalised promotion strategies for a union based on performance, event participation, and member demographics. Helps unions improve outreach effectiveness.
Operation	Signature	recommendTargetUnions(eventId: String): String
	Function	Suggests unions that are statistically most likely to engage with a particular event, helping optimise event visibility and union collaboration opportunities.
Operation	Signature	recommendResourceAllocation(): String
	Function	Produces AI-driven recommendations on distributing resources (e.g., promotional budget, event materials) based on predicted impact and historical outcomes, supporting efficient system-wide decision-making.

Task 5a: Structural Model



This class diagram models the internal structure of the UnionMembershipService, focusing on MembershipApproval. Attributes are encapsulated and operations match 4a ensuring consistency. Associations and multiplicities follow the logical workflow or retrieving, evaluating and persisting membership decision records.

5.b) Behavioural Diagram:


MembershipApprovalProcessor Internal Behaviour
:MembershipApproval
:MembershipApprovalProcessor
:MembershipApplicationRepository
:MembershipApplication
:Descision Record
:Notification
:UnionDataSync
alt
[applicationFound == true]
rejectMembership(unionId: String, officerId: String, reason: String): String
rejectMembership(unionId: String, officerId: String, reason: String): String
findApplicationByUnionId(unionId: String): MembershipApplication
application
DecisionRecord
decisionId
saveDecisionRecord(decisionRecord)
saved
updateApplication(application)
updated
alt
[notificationsPermitted == true]
notifyUnionOfficers("membership rejected")
notifyMembrshipDecision(unionId, "rejected")
syncMembershipDecision(unionId, "rejected")
service response
service response membership rejected
[else]
service response
membership rejected
[applicationFound == false]
service response
error message membership not found

At the start of the COMP5047 coursework, our group formed quickly and set up a WhatsApp chat along with a weekly meeting time on Tuesdays at around 15:00. This helped us stay in regular contact and made it easy to check progress and share updates. I took responsibility early on for creating the shared GitHub repository and helping organise the first few discussions about how we would divide the work. This made it easier for everyone to upload their diagrams and to keep track of what needed to be done.

In terms of project management, I often found myself guiding the group's workflow. I reminded the team about deadlines, encouraged everyone to upload their work in stages rather than at the last minute, and kept an eye on how the different subsystems would eventually fit together. Whenever the group needed clarification about a requirement or a diagram, I helped summarise the decisions we made so that they were recorded clearly in our meeting notes. This ensured that we remained consistent across the whole project. There were moments when participation dipped slightly, and I stepped in to keep the group moving by checking what work was outstanding and making sure nobody was left unsure of their tasks.

My technical contribution went beyond completing the USU Operation System. I also contributed to the shared architectural work across the group. For example, I helped confirm how membership approval data should move across subsystems and made sure that the interfaces described in Task 4 matched the later diagrams in Task 5. When reviewing the work of 19238600 and 19323083, I checked that their class diagrams and sequence diagrams aligned with the structural and behavioural rules required by the case study. This made our overall architecture more coherent. I also spent time understanding how my subsystem depended on theirs, especially for decision records and notification flows, so that our combined diagrams made sense as a whole system.

Working with 19238600 and 19323083 felt smooth and productive. They contributed their work on time, responded clearly in discussions, and were open to revising their diagrams whenever we spotted inconsistencies during meetings. Our communication throughout the project was straightforward, and we rarely needed long debates to resolve differences. Even when we had different interpretations of a requirement, we were able to talk it through and agree on a shared solution quickly. This helped create a positive and cooperative working environment.

Overall, this coursework strengthened my project management skills and improved my confidence when working with larger system models. It also helped me understand the value of clear communication and consistent documentation when working in a team. The experience showed me how important it is to align subsystem designs early so the final integrated architecture feels coherent. I believe my contribution to organising the group, supporting shared technical decisions, and maintaining communication helped the team produce a strong and well-integrated set of deliverables.